ARP Spoofing and Packet Sniffing Attack Using Bettercap

A Man-in-the-Middle Attack Demonstration in a Controlled Lab Environment

Prepared by:

Ibrahim Ayomide Fayomi

Cybersecurity Analyst

fayomiibrahim44@gmail.com

https://www.linkedin.com/in/fayomi31/

Date: April 1, 2026

Environment: Kali Linux / Oracle VirtualBox / Windows 10

Abstract

This report documents the practical demonstration of an ARP spoofing and packet sniffing attack carried out in a controlled virtual lab environment. The attacker machine ran Kali Linux using the Bettercap framework version 2.41.5. The victim machine was a Windows 10 installation hosted in Oracle VirtualBox. By poisoning the ARP cache of the target device at IP address 172.20.10.6, the attacker positioned itself between the victim and the gateway, enabling full interception of all network traffic in both directions. The captured data included DNS queries revealing every domain the victim resolved, HTTPS Server Name Indication (SNI) headers revealing the destination of encrypted connections, and complete plaintext HTTP responses. Real websites were visited on the victim machine, including Google, Yahoo, Tesla, and Facebook, during the live attack. The results confirm that devices on shared or unsecured networks are exposed to passive and active traffic interception without any interaction from the victim. This work is intended for educational purposes only, executed strictly in an isolated lab setting.

Introduction

The Address Resolution Protocol (ARP) was designed in the early days of networking to resolve IP addresses to physical MAC addresses on a local network segment. It was built without any form of authentication or verification. A device on the network can send an ARP reply at any time, even when no request was made, and other devices will update their ARP caches accordingly. This design choice, reasonable in trusted networks of the 1980s, creates a serious vulnerability in modern multi-device environments.

ARP spoofing, also called ARP poisoning, exploits this weakness by sending forged ARP reply packets to both a target device and the local gateway. The target is told that the attacker's MAC address belongs to the gateway. The gateway is told the attacker's MAC address belongs to the target. As a result, all traffic between the target and the internet flows through the attacker's machine, creating a Man-in-the-Middle (MITM) position.

Once in the MITM position, the attacker can passively read all traffic using a packet sniffer. This lab demonstrates the complete attack chain from network discovery to live traffic interception, using only the tools built into the Bettercap framework. The goal is to show how simple this attack is to execute, and why defenders need to understand it.

Project Objectives

The specific objectives of this project are as follows:

- Launch Bettercap on the Kali Linux attacker machine targeting the eth0 network interface
- Use the net.probe module to actively discover all live hosts on the 172.20.10.0/28 subnet
- Identify the Windows 10 victim machine at IP address 172.20.10.6 from the ARP table
- Configure and activate the arp.spoof module with full duplex mode to perform MITM positioning
- Activate the net.sniff module to capture and analyze live traffic from the victim
- Demonstrate DNS query interception, HTTPS SNI leakage, and HTTP response capture
- Document real-world browsing behavior on the victim machine and correlate it with captured data
- Explain the security implications and available defenses

Lab Environment

The attack was conducted entirely within a private virtual lab. No external or production systems were involved.

Component	Details
Attacker Operating System	Kali Linux (Desktop, terminal interface)
Victim Operating System	Windows 10 (hosted in Oracle VirtualBox)
Network Range	172.20.10.0/28
Gateway IP	172.20.10.3 (MAC: 08:00:27:ad:c7:68)
Attacker Interface	eth0

Target IP Address	172.20.10.6 (MAC: 08:00:27:85:1d:78)
Additional Device	172.20.10.2 (Intel Corporate)
Additional Device	172.20.10.4 (HR-PC, PCS Systemtechnik GmbH)
Bettercap Version	2.41.5 (built for linux amd64 with go1.24.9)
Virtualization	Oracle VirtualBox
Date of Execution	April 1, 2026

Tools and Technologies

The following tools and modules were used during this project:

Tool / Module	Purpose
Bettercap v2.41.5	Main framework. Integrates network discovery, ARP spoofing, and packet sniffing into a single interactive environment.
net.probe	Actively probes the subnet by sending dummy UDP packets to all 16 possible IPs in the /28 range. Forces live hosts to reveal themselves.
net.recon	Passive reconnaissance module. Auto-starts when net.probe is activated. Monitors ARP traffic and maintains a list of known endpoints.
arp.spoof	Sends forged ARP replies to the target and the gateway. Redirects traffic through the attacker machine, creating the MITM position.
net.sniff	Captures and parses all network packets that flow through the attacker machine. Decodes DNS, HTTP, HTTPS (SNI), and other protocols in real time.
Oracle VirtualBox	Hosts the Windows 10 victim machine in an isolated virtual environment on the same host as the Kali attacker.
Microsoft Edge	Browser used on the victim machine to generate network traffic during the attack demonstration.

Methodology

The attack was executed in a clear, reproducible sequence. Each step is documented below with the corresponding terminal screenshot and a technical explanation of every command and its output.

Step 1: Launching Bettercap and Opening the Manual

The first action taken on the Kali Linux machine was opening a terminal on the Desktop and running the following command:

```
sudo bettercap -iface eth0
```

This command starts the Bettercap framework and binds it to the eth0 network interface. The sudo prefix grants the root-level privileges that are required for raw socket access and ARP manipulation. Bettercap immediately identifies the network range as 172.20.10.0/28 and detects the gateway at 172.20.10.3. The framework version is 2.41.5, compiled for Linux amd64 with Go 1.24.9.

After startup, the help command is entered to display the command manual. This shows all general commands available within the Bettercap interactive session: help MODULE for module-specific documentation, set NAME VALUE to configure parameters, active to list running modules, and quit to exit the session. The gateway monitor starts immediately in the background, logging the event with [sys.log] [inf] gateway monitor started.

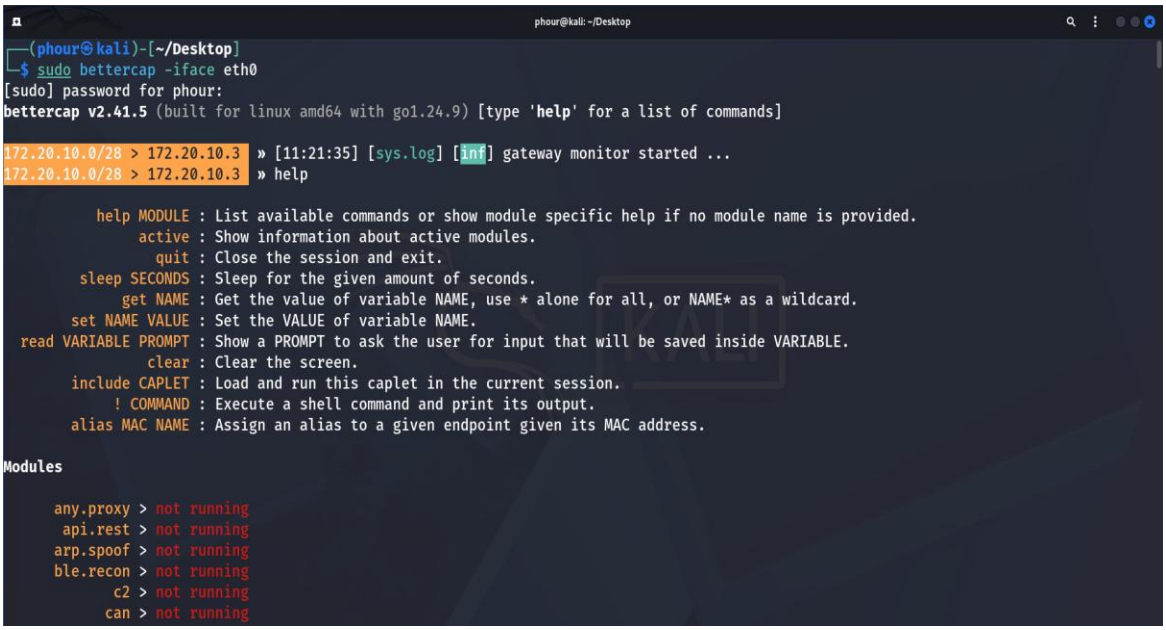

Figure 1. Bettercap launched on eth0, help manual displayed, gateway monitor active.

Step 2: Reviewing Available Modules

Entering the help command without any module name displays a full list of all built-in Bettercap modules, each shown with a status of "not running." This provides a complete view of the attack surface available within Bettercap.

The modules relevant to this project are highlighted in the screenshot: arp.spoof for ARP poisoning and MITM positioning, net.probe for active host discovery, net.recon for passive background monitoring, and net.sniff for live packet capture. Other modules visible include any.proxy, api.rest, ble.recon, c2, dhcp6.spoof, dns.proxy, dns.spoof, http.proxy, https.proxy, mac.changer, and more. This demonstrates the breadth of Bettercap as a full-featured network attack framework.

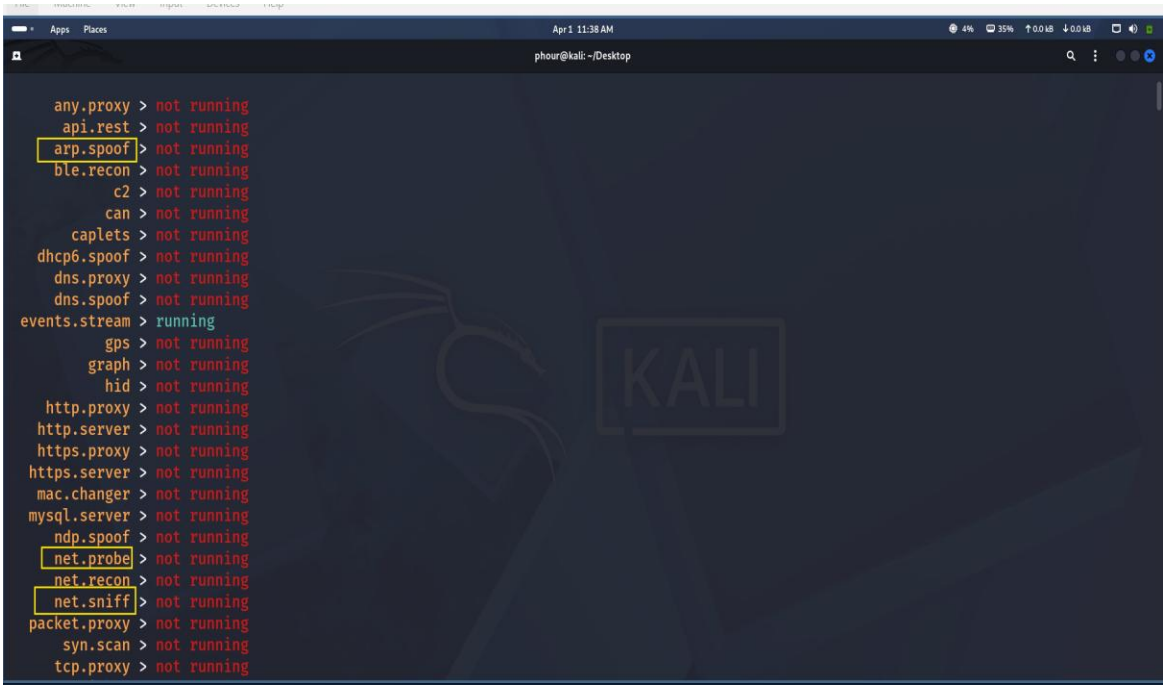

Figure 2. Complete list of Bettercap modules, all not running. Highlighted: arp.spoof, net.probe, net.recon, net.sniff.

Step 3: Network Discovery with net.probe

Before launching the attack, the network must be scanned to identify all live hosts and select the target. The following commands are run:

```
help net.probe
```

```
net.probe on
```

The help net.probe command displays the module manual. It explains that net.probe sends dummy UDP packets to every IP in the subnet to discover live hosts. The module has configurable parameters including net.probe.mdns (mDNS discovery, default true), net.probe.nbns (NetBIOS name service, default true), net.probe.throttle (rate limiting in milliseconds, default 10), net.probe.upnp (UPnP discovery, default true), and net.probe.wsd (WSD discovery, default true).

Once net.probe on is executed, Bettercap begins scanning all 16 addresses in the 172.20.10.0/28 range. The output logs show net.recon starting automatically as a dependency. Within seconds, live hosts are detected and logged: 172.20.10.2 (Intel Corporate vendor, 84 kB sent, 88 kB received), 172.20.10.4 and 172.20.10.5 (both PCS Systemtechnik GmbH). IPv6 link-local addresses are also resolved for these devices.

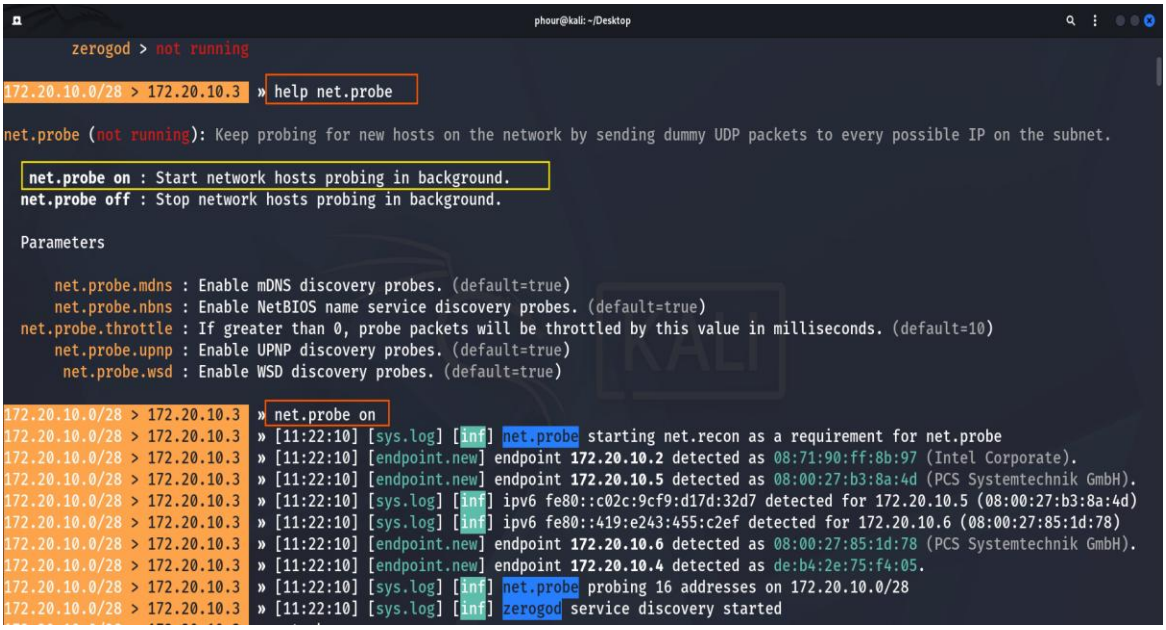

Figure 3. net.probe help manual shown, then activated. Three live hosts discovered on the /28 subnet.

Step 4: Viewing the ARP Table and Identifying the Target

With live hosts discovered, the next step is to view the ARP table and select the target. The net.show command is used:

```
net.show
```

This command outputs a formatted table of all discovered devices with their IP addresses, MAC addresses, vendor names, traffic statistics, and last seen timestamps. The full ARP table shows:

IP	MAC Address	Vendor	Role
172.20.10.3	08:00:27:ad:c7:68	PCS Systemtechnik GmbH	Gateway (eth0)
172.20.10.2	08:71:90:ff:8b:97	Intel Corporate	Second device on subnet
172.20.10.4	de:40:2e:f5:f4:05	PCS Systemtechnik GmbH	HR-PC
172.20.10.6	08:00:27:85:1d:78	PCS Systemtechnik GmbH	TARGET - Windows 10 victim

The target device at 172.20.10.6 has already generated 1.8 MB of received traffic, indicating active internet use. This confirms it is the Windows 10 virtual machine. The help arp.spoof command is then entered to review the ARP spoofing module options before configuring the attack.

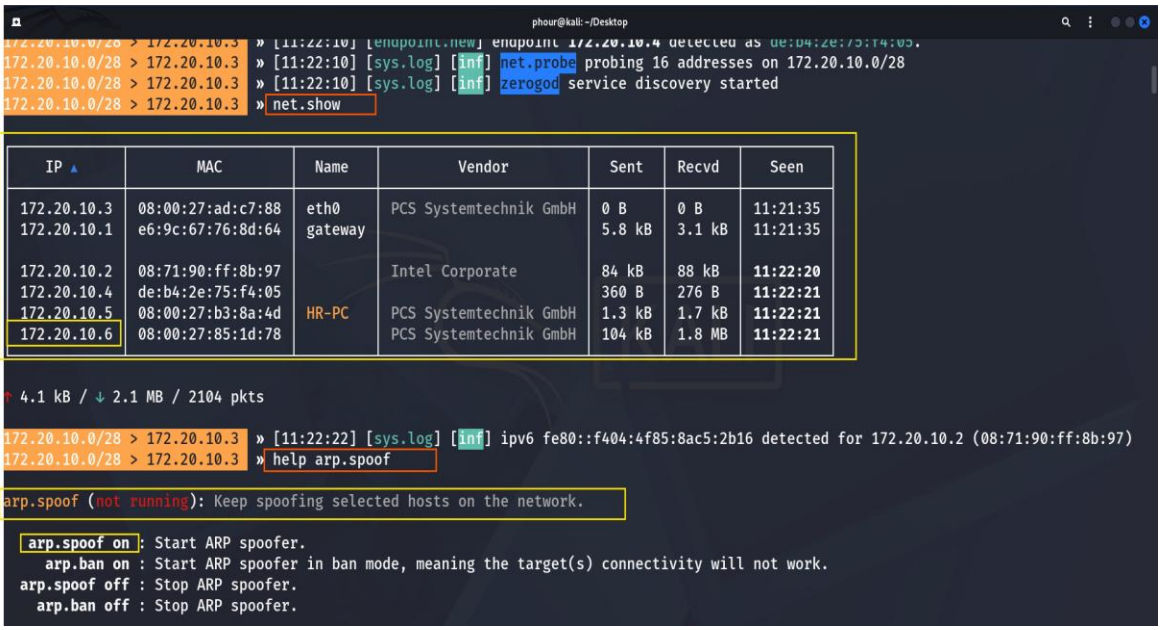

Figure 4. net.show ARP table output. Target 172.20.10.6 identified. arp.spoof help manual shown.

Step 5: Configuring ARP Spoof and Starting the MITM Attack

The arp.spoof module is configured with two specific parameters before activation:

```
set arp.spoof.fullduplex true
set arp.spoof.targets 172.20.10.6
arp.spoof on
```

The arp.spoof.fullduplex parameter, when set to true, instructs Bettercap to poison both the target machine and the gateway simultaneously. Without full duplex mode, only the target's ARP cache is poisoned, meaning only traffic from the victim to the internet is redirected. The gateway still sends responses directly to the victim. Full duplex ensures that return traffic also passes through the attacker, giving complete bidirectional visibility. Bettercap logs a warning at this point: "full duplex spoofing enabled, if the router has ARP spoofing protection mechanisms, the attack will fail."

The arp.spoof.targets parameter restricts the poisoning to only 172.20.10.6. Without this setting, Bettercap defaults to spoofing the entire subnet, which would generate significantly more traffic and potentially disrupt all devices on the network. Targeting a single IP is both more controlled and more realistic in a real attack scenario.

When arp.spoof on is executed, Bettercap logs: "[inf] arp.spoof: starting, probing 1 targets." It begins sending forged ARP reply packets to the victim and the gateway at regular intervals, keeping the ARP caches poisoned.

Immediately after, net.sniff on is activated to begin capturing all traffic flowing through the attacker machine:

```
net.sniff on
```

The first intercepted packets appear within seconds: an HTTP request from 172.20.10.6 to an external server at 196.49.32.6, followed by an HTTP 206 Partial Content response. This confirms the MITM position is active and traffic is being intercepted.

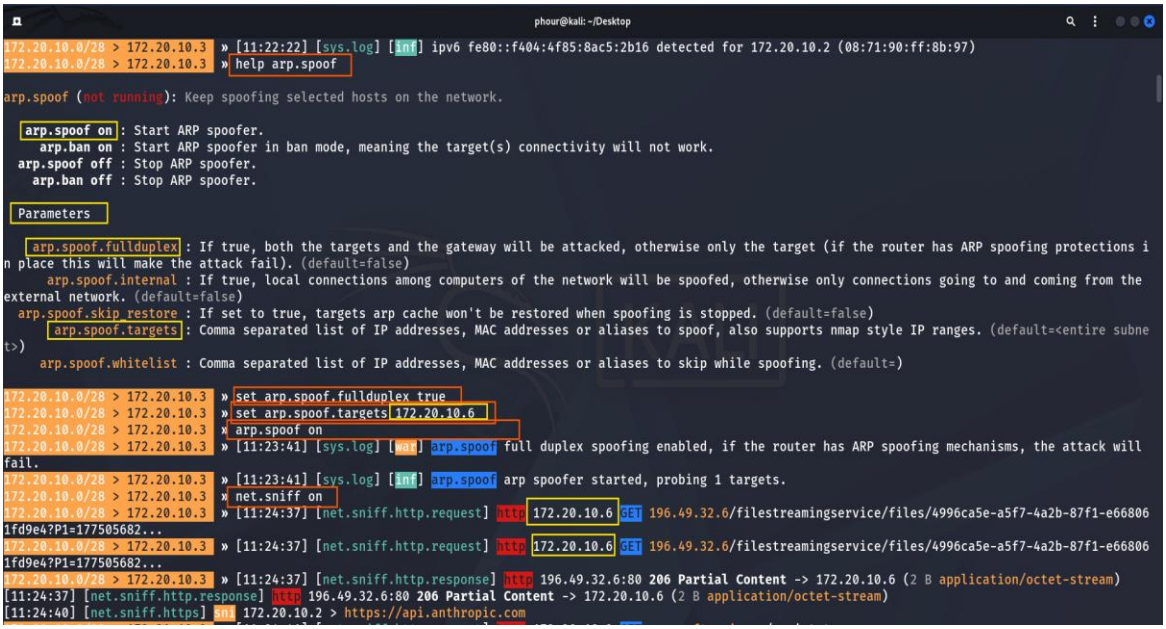

Figure 5. arp.spoof configured (fullduplex=true, target=172.20.10.6), MITM started, net.sniff activated, first packets captured.

Step 6: Live Traffic Capture - Initial DNS and HTTPS Traffic

With net.sniff active and the MITM position established, the terminal begins logging all intercepted traffic in real time. The first wave of captured data shows a mix of DNS queries, HTTPS SNI headers, and HTTP responses.

DNS queries appear labeled as [net.sniff.dns]. These are completely unencrypted. The attacker sees the exact domain name being queried and the DNS server's response including the resolved IP address. In this capture window, the following domains are resolved: gmail.com resolving to 162.250.74.80, ax-002.a-msedge.net resolving to 150.171.27.11 (Microsoft Edge CDN), yahoo.com resolving to 98.137.11.163 and 74.6.143.20, and multiple queries for facebook.com subdomains.

HTTPS connections appear labeled as [net.sniff.https]. These entries show the destination domain from the TLS Server Name Indication (SNI) field. The attacker cannot read the content of HTTPS sessions, but the SNI reveals exactly which site is being visited. Connections to edge.microsoft.com and accounts.google.com are visible in this window.

An HTTP response labeled [net.sniff.http.response] shows a 200 OK from 23.200.147.185, which is a content delivery network address associated with Microsoft services. The response headers are fully readable, including Content-Type and Cache-Control.

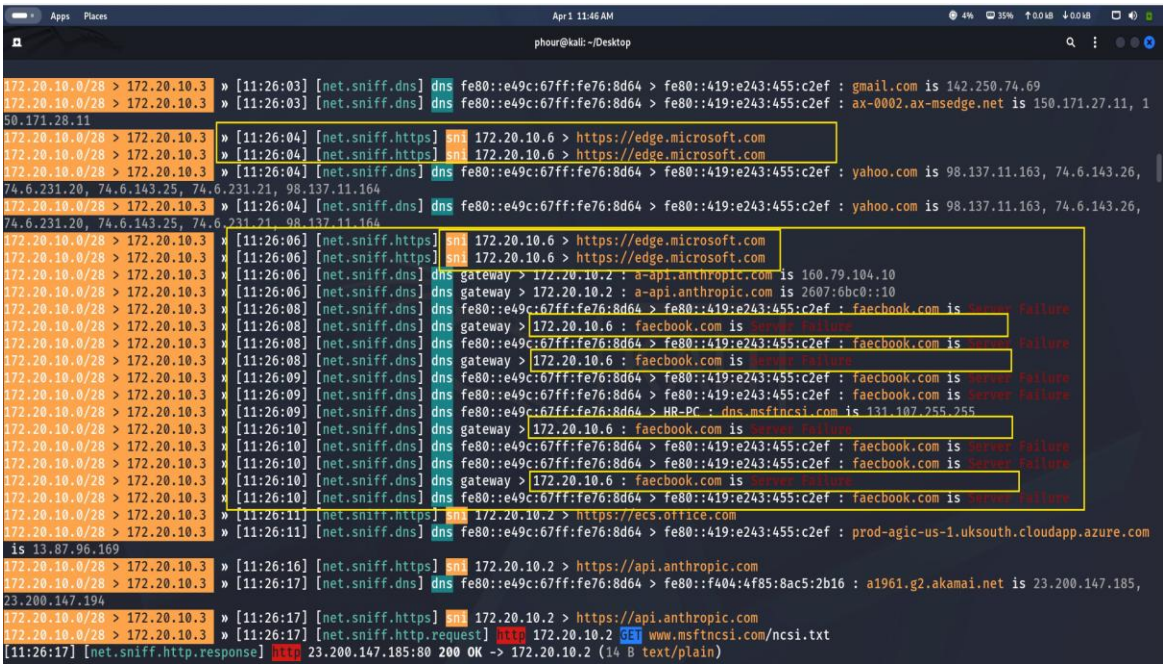

Figure 6. Live packet sniffing active. DNS queries, HTTPS SNI, and HTTP responses from target 172.20.10.6 all visible.

Step 7: Continued Sniffing - Facebook DNS Failures and Microsoft Services

As sniffing continues, the logs show repeated DNS queries for facebook.com subdomains. These queries are logged but the responses show failure or unusual behavior, matching what is observed on the victim machine's browser: Facebook fails to load.

Multiple entries in this window show the victim resolving Microsoft infrastructure: accounts.google.com resolving to 172.217.72.84, youtube.com, edge.microsoft.com, and clients2.google.com. Windows OS background services also generate traffic, including client.wns.windows.com (Windows Notification Service) and clients.sms.windows.com.

The distinction between intentional user browsing and background OS traffic is visible in the logs. User-initiated DNS queries tend to cluster around the domains the victim was actively navigating to (Google, Yahoo, Tesla, Facebook), while background traffic from Windows services runs continuously regardless of what the user is doing.

Figure 7. Continued packet sniffing. Facebook DNS failures visible. Microsoft OS background services captured alongside user browsing traffic.

Step 8: Victim Browsing Google, Session Disruption Observed

The victim machine screenshot shows the Windows 10 VirtualBox window with Microsoft Edge open. The browser is at accounts.google.com/info/sessionExpired, displaying the message: "You're not signed in. Your session ended because there was no activity. Try signing in again."

This is a direct consequence of the MITM attack. When the attacker's machine sits between the victim and Google's servers, the network path changes. Google's session validation systems detect this change in the connection route and invalidate the existing session token for security reasons. The victim's Google account session is terminated without any action on their part.

On the attacker's terminal, the sniff logs show a cluster of mDNS browsing queries from the victim: [zeroconf.browsing] entries showing the target is querying for _googlecast._tcp.local services, which are Chromecast device discovery requests from the Chrome browser. These mDNS queries reveal that the victim is running Google Chrome or a Chromium-based browser on their Windows machine.

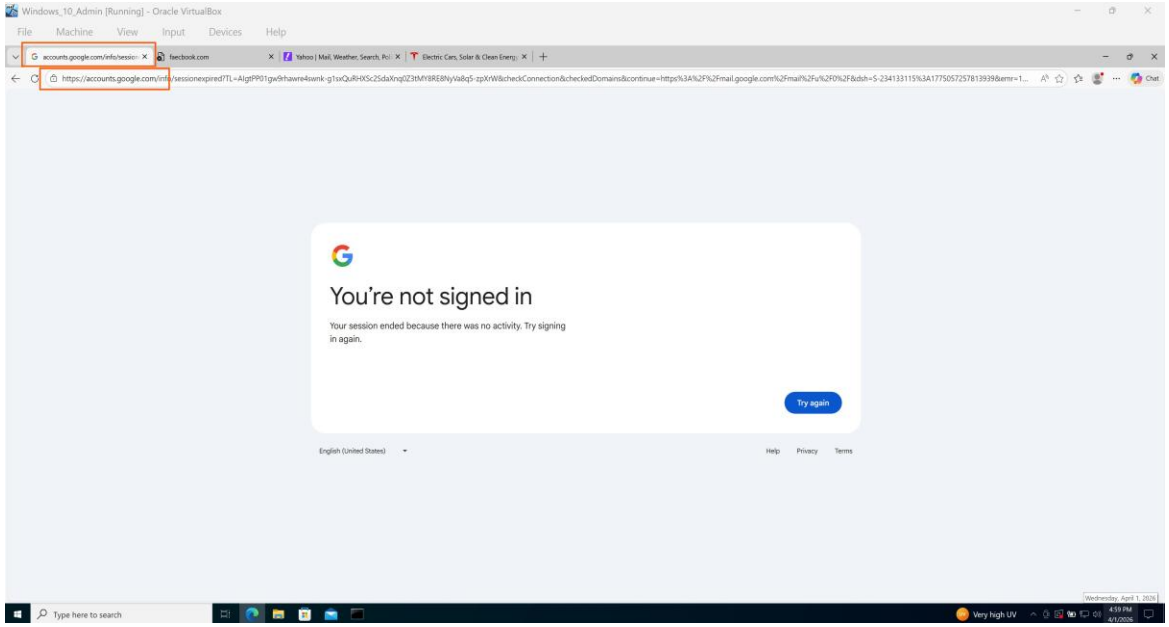

Figure 8. Victim machine (Windows 10 in VirtualBox) showing Google session expired due to MITM attack interference.

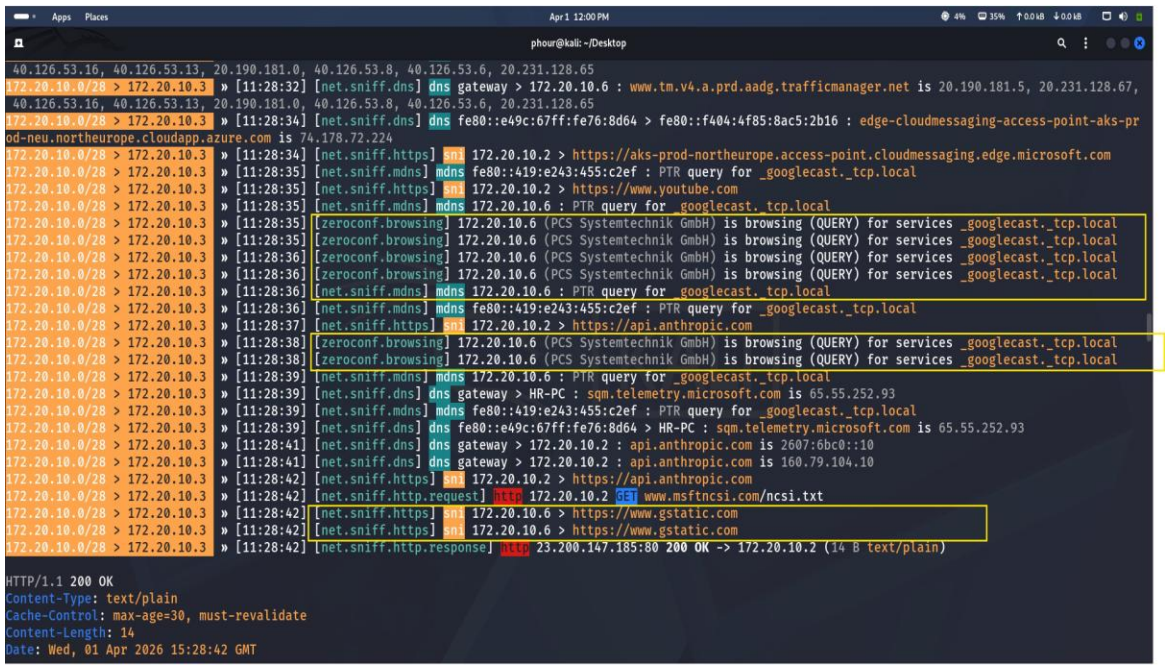

Figure 9. Attacker terminal showing mDNS Googlecast queries from victim, Google service DNS resolutions, and Microsoft NCSI 200 OK response captured.

Step 9: Tesla and Yahoo Browsing Sniffed

The victim then navigates to Tesla.com and Yahoo.com. On the victim machine, the Tesla website loads successfully, showing the Tesla homepage with the "Full Self-Driving (Supervised)" hero image. The Yahoo homepage also loads, showing the news feed with a notification prompt.

On the attacker machine, the sniff logs immediately reflect this activity. For Tesla, multiple HTTPS SNI entries appear: tesla.com, location-services-prd.tesla.com, digitalassets.tesla.com, and CDN addresses including Akamai edge nodes (a1792.dsca.akamai.edge.net, e0530.dsca.akamai.edge.net). DNS resolution for tesla.com reveals the IP addresses used by Tesla's CDN infrastructure: 3.18.50.207, 23.7.244.287, and related AWS/Akamai addresses.

For Yahoo, the DNS resolution shows yahoo.com mapping to 98.137.11.163 and 74.6.143.20. HTTPS SNI entries confirm connections to yahoo.com and related services. The attacker knows with certainty that the victim is visiting Tesla and Yahoo, including which specific subdomains and CDN endpoints are being contacted, all without breaking any encryption.

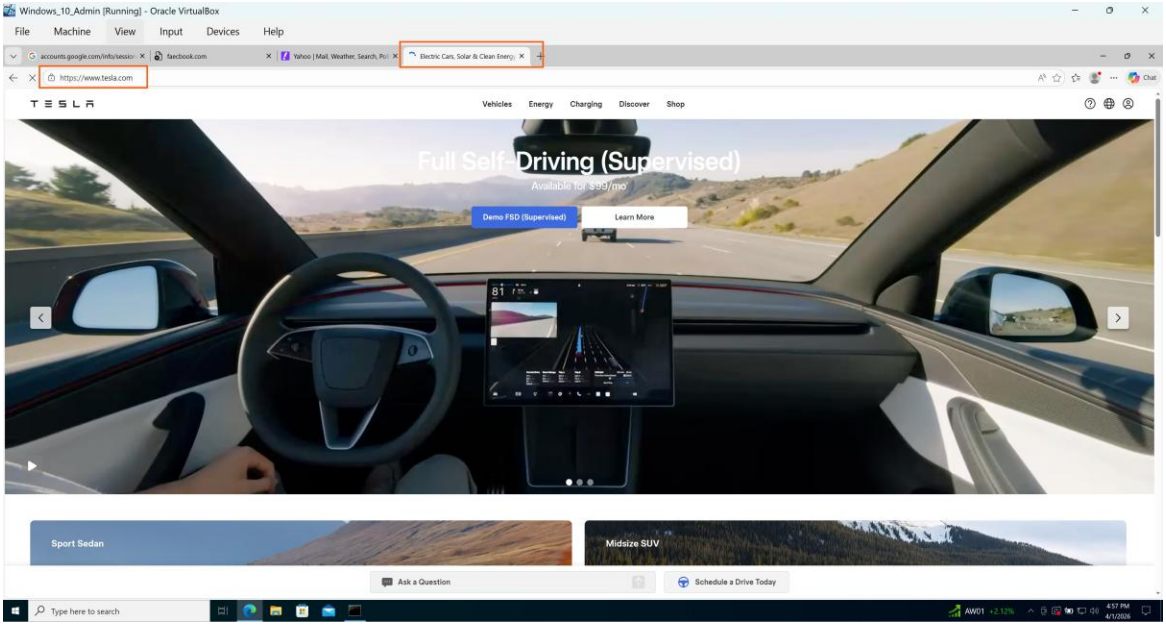

Figure 10. Tesla website loaded successfully on victim machine (Windows 10 in VirtualBox).

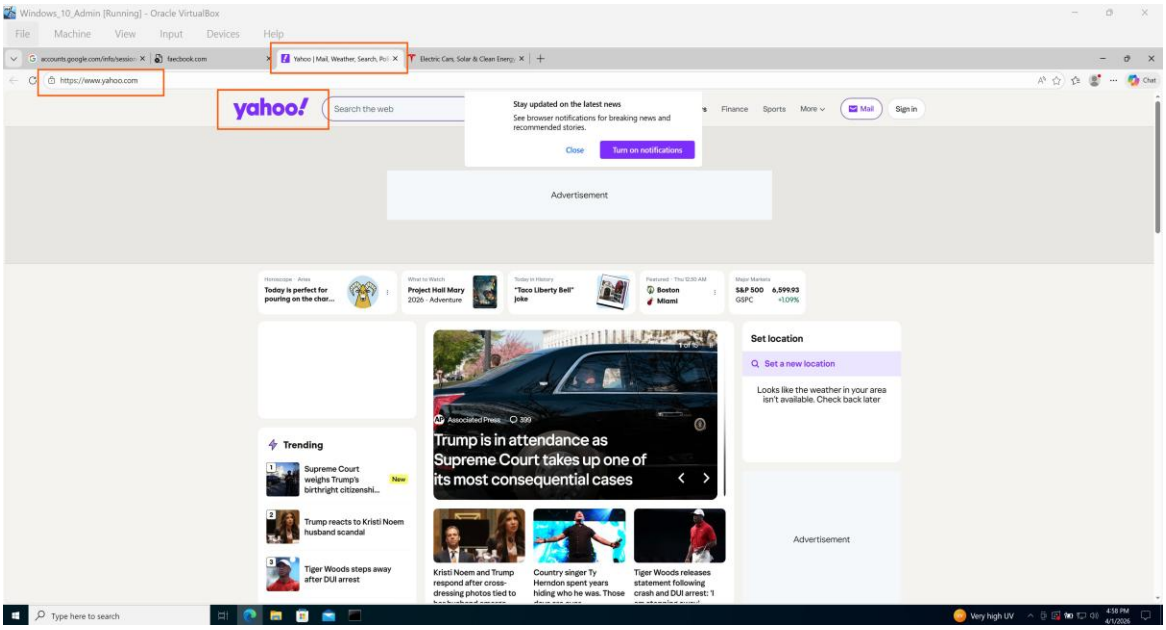

Figure 11. Yahoo website loaded on victim machine showing news homepage.

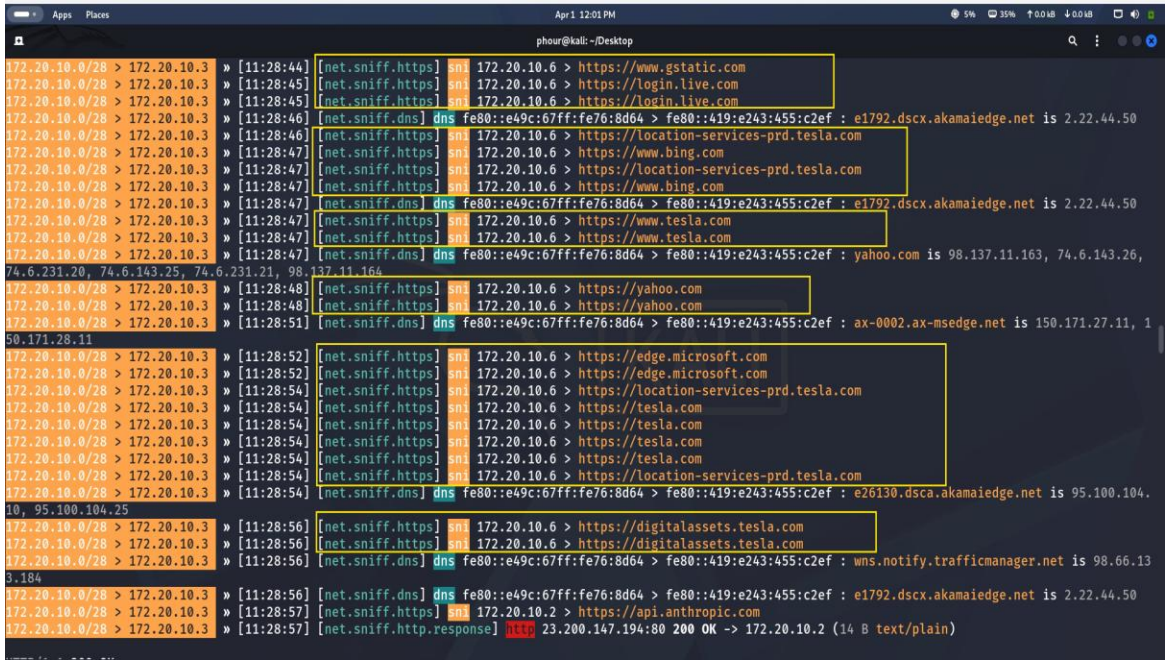

Figure 12. Attacker sniff log showing Tesla CDN DNS resolutions, Yahoo HTTPS connections, and Microsoft edge services from 172.20.10.6.

Step 10: Facebook DNS Failure on Victim Machine

The victim attempts to visit faecbook.com (typed with a typographic error in the browser address bar). The Windows 10 machine displays an error in Microsoft Edge: "Hmmm... can't reach this page" with the suggestion "Did you mean http://facebook.com/?" and the error code ERR_PROBE_FINISHED_NXDOMAIN.

The NXDOMAIN error means the DNS lookup returned no result for faecbook.com, which is correct behavior since that domain does not exist. However, the attacker's sniff logs from this same period show repeated DNS queries for facebook.com (the correct domain) also returning unusual or failed results. This indicates that DNS resolution for Facebook's actual domain was experiencing instability during this phase of the attack, likely due to network disruption caused by the ARP cache poisoning affecting DNS traffic routing.

This is an important observation. ARP spoofing does not intentionally block any site, but the rerouting of traffic can introduce latency, packet loss, and DNS resolution failures, particularly for services with complex DNS configurations or those that check for network path anomalies.

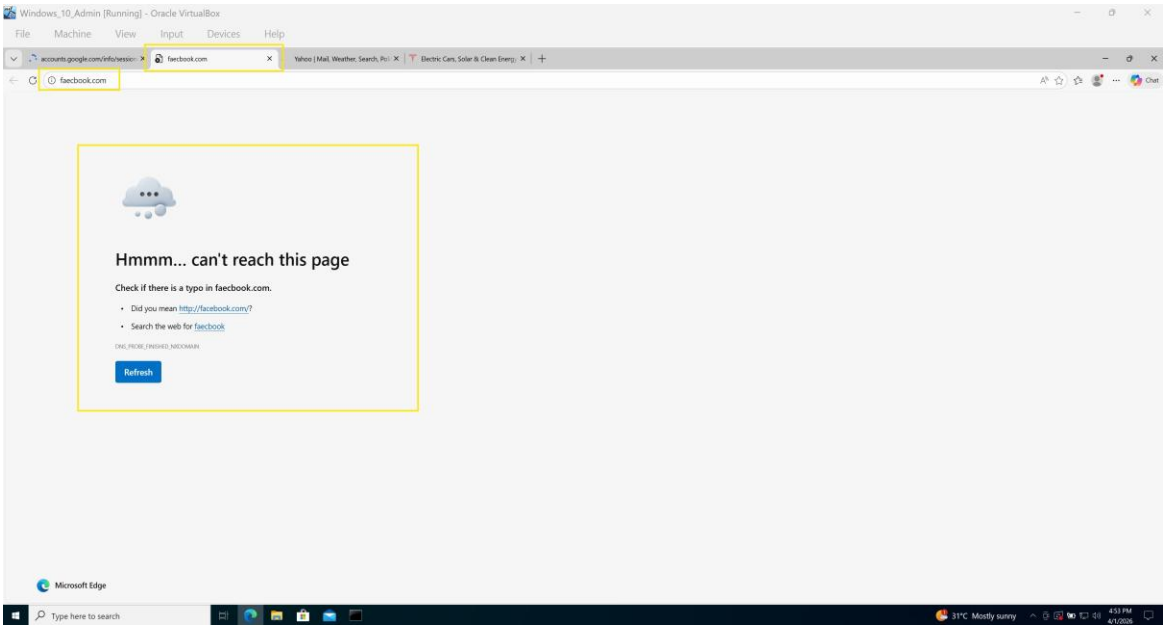

Figure 13. Victim machine showing Facebook page load failure. Browser error: ERR_PROBE_FINISHED_NXDOMAIN.

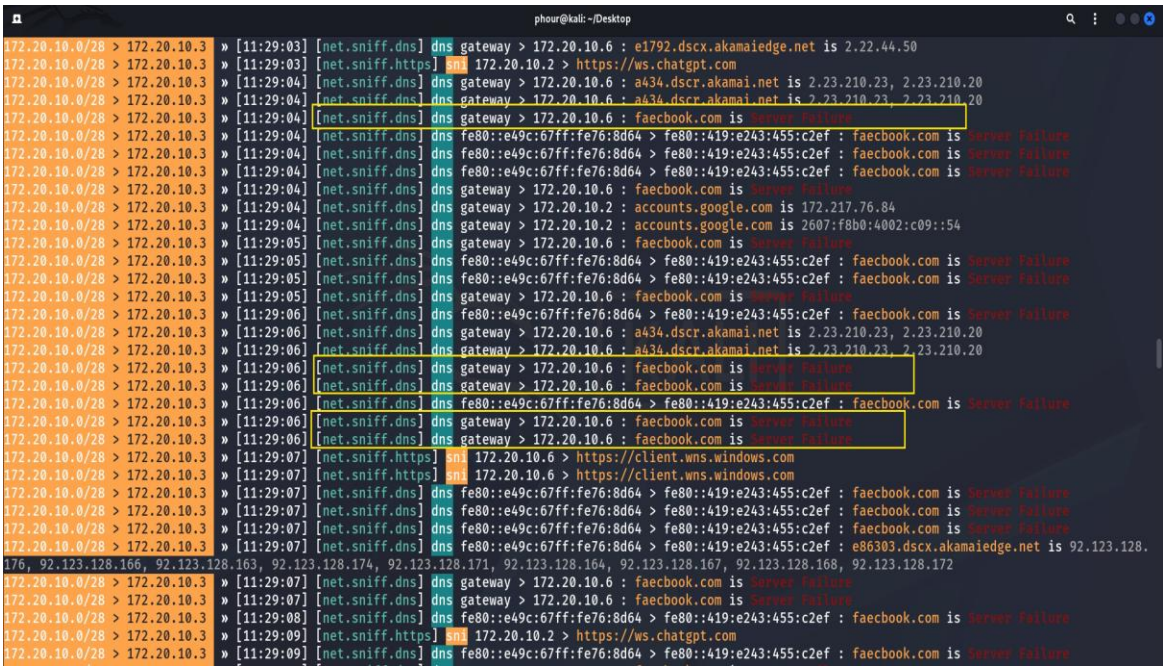

Figure 14. Attacker sniff log showing repeated facebook.com DNS failures alongside continued capture of other services including ChatGPT and Windows system traffic.

Step 11: Deep Google Services and Additional Traffic Interception

As the sniffing session continues, more Google infrastructure becomes visible in the logs. DNS queries and HTTPS SNI entries appear for accounts.google.com, mail.google.com, play.google.com, clients.l.google.com, and other Google service endpoints. The attacker can see that the victim is accessing Gmail, indicated by the HTTPS connection to mail.google.com.

Third-party analytics and tracking services also appear: googletagmanager.com resolves to 142.251.210.4, revealing that the web pages the victim is visiting use Google Tag Manager for tracking. Optimizely CDN addresses appear (cdn.optimizely.com, a271064i0188.cdn.optimizely.com), indicating at least one website the victim is browsing uses A/B testing tools.

Grammarly browser extension traffic is captured, with connections to f-log-extension.grammarly.io and gateway.grammarly.com. This reveals that the victim has the Grammarly extension installed in their browser. Twitter/X CDN traffic appears via static.ads-twitter.com. WhatsApp web connections are also captured, showing the victim has WhatsApp Web open: mes-ds.cdo.whatsapp.net resolves to 102.132.101.60 and web.whatsapp.com HTTPS connections appear.

A Windows Update download is also captured: the device at 172.20.10.4 (HR-PC) sends a request to au.download.windowsupdate.com, which initiates a download of a Windows update package. This traffic was not triggered by any user action but by the Windows Update background service.

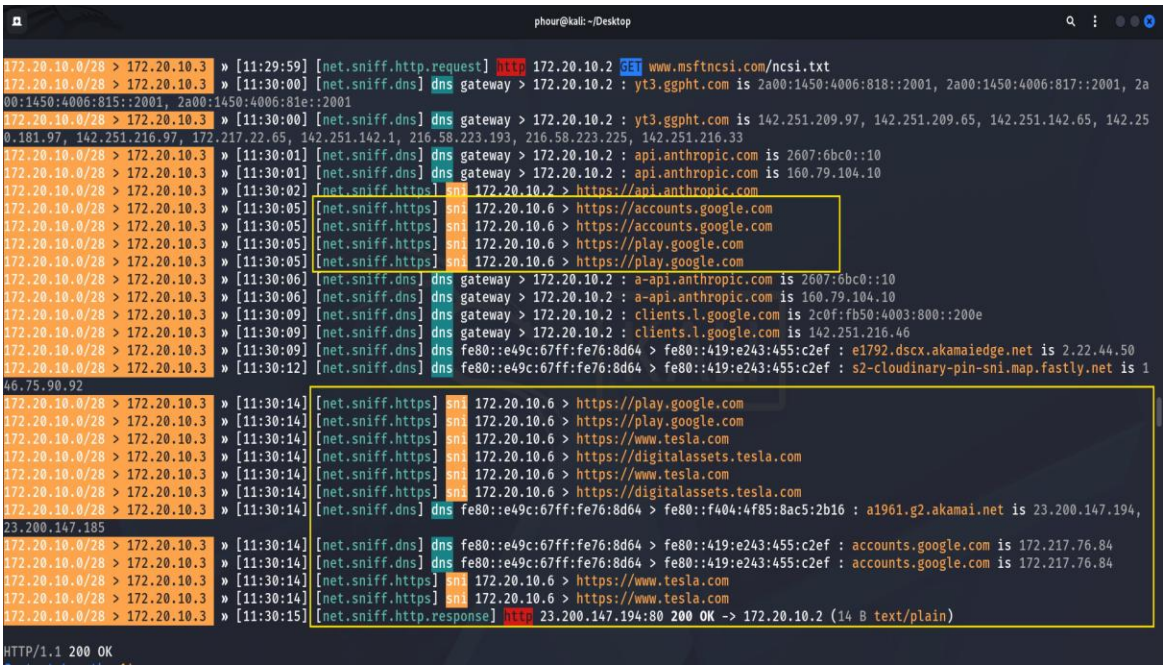

Figure 15. Sniff log showing deep Google service traffic: accounts.google.com, play.google.com, clients.l.google.com, and Tesla CDN endpoints.

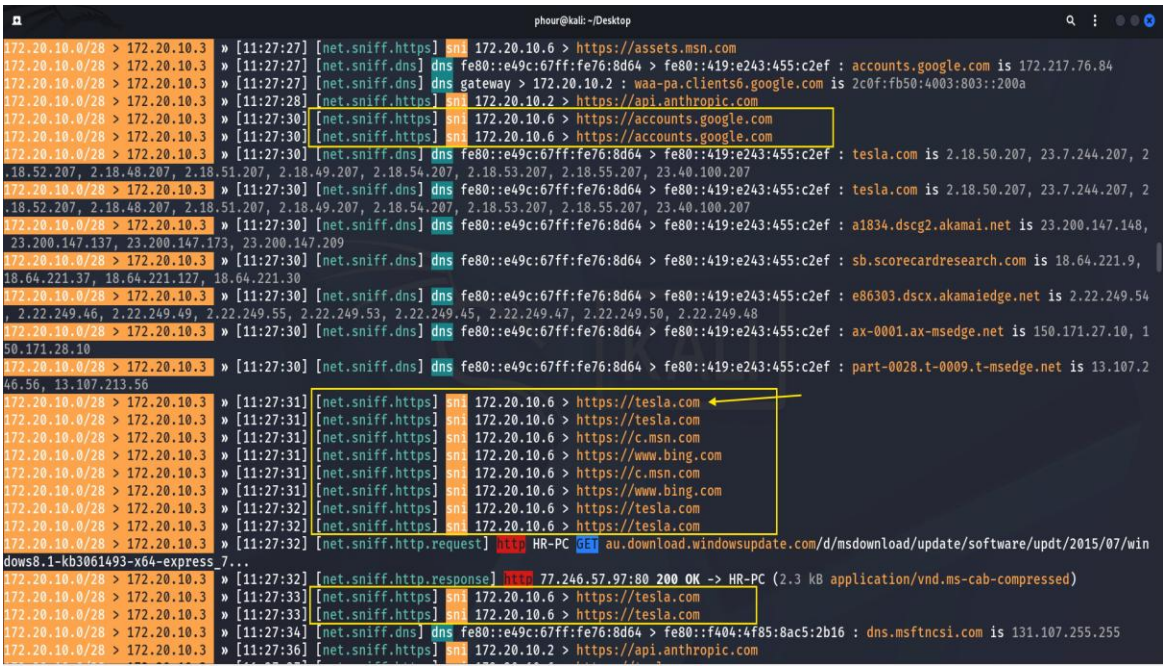

Figure 16. Sniff log showing Tesla.com DNS resolution to CDN IPs, MSN and Bing services, and Windows update download initiated by HR-PC.

Step 12: Final Sniffing Stage and Session Termination

In the later phase of the sniffing session, the logs show Windows.msn.com traffic, MSN news services, and additional WhatsApp web connections. The Microsoft NCSI check (msftncsi.com/ncsi.txt) continues to run at regular intervals, always returning HTTP/1.1 200 OK. This is a Windows feature that periodically checks network connectivity by fetching a small text file.

To end the session, the quit command is used. Bettercap prompts for confirmation: "Are you sure you want to quit this session? y/n". Upon confirming, the framework stops net.sniff and begins the ARP cache restoration process: "[net.sniff...] waiting for ARP spoofer to stop... restoring ARP cache of 1 targets." This is an important step. Bettercap sends corrective ARP replies to both the victim and the gateway, restoring the correct MAC address mappings. Without this cleanup step, the victim's ARP cache would remain poisoned until it times out naturally, continuing to route traffic through the attacker's now-offline machine, which would break the victim's internet access.

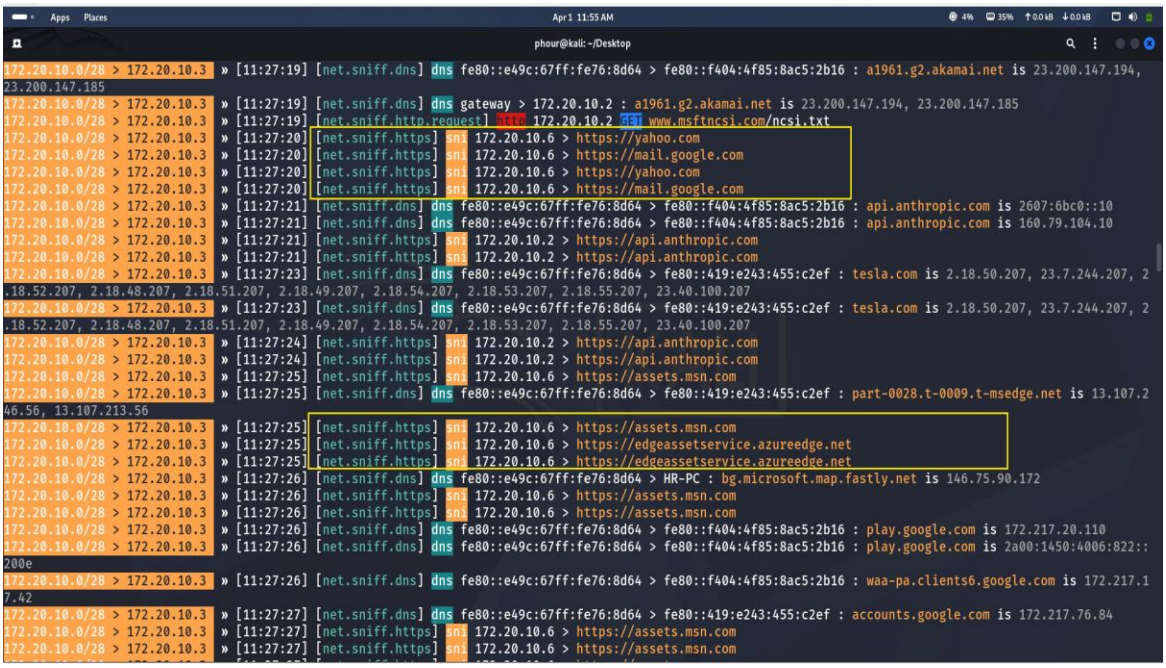

Figure 17. Attacker sniff log showing Yahoo and Google HTTPS traffic, MSN services, and CDN connections.

```
phour@kali: ~/Desktop
Content-Type: text/plain

Microsoft NCSI

172.20.10.0/28 > 172.20.10.3  » [11:31:48] [net.sniff.http.request] http 172.20.10.2 GET www.msftncsi.com/ncsi.txt
172.20.10.0/28 > 172.20.10.3  » [11:31:50] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::419:e243:455:c2ef : www.googletagmanager.com is 142.251.216.4
0
172.20.10.0/28 > 172.20.10.3  » [11:31:50] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::419:e243:455:c2ef : www.googletagmanager.com is 142.251.216.4
0
172.20.10.0/28 > 172.20.10.3  » [11:31:50] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::419:e243:455:c2ef : e1792.dscx.akamaiedge.net is 2.22.44.50
172.20.10.0/28 > 172.20.10.3  » [11:31:51] [net.sniff.https] sni 172.20.10.6 > https://cdn.optimizely.com
172.20.10.0/28 > 172.20.10.3  » [11:31:51] [net.sniff.https] sni 172.20.10.6 > https://cdn.optimizely.com
172.20.10.0/28 > 172.20.10.3  » [11:31:52] [net.sniff.https] sni 172.20.10.6 > https://www.googletagmanager.com
172.20.10.0/28 > 172.20.10.3  » [11:31:52] [net.sniff.https] sni 172.20.10.6 > https://www.googletagmanager.com
172.20.10.0/28 > 172.20.10.3  » [11:31:52] [net.sniff.https] sni 172.20.10.6 > https://www.googletagmanager.com
172.20.10.0/28 > 172.20.10.3  » [11:31:52] [net.sniff.https] sni 172.20.10.6 > https://www.googletagmanager.com
172.20.10.0/28 > 172.20.10.3  » [11:31:55] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:31:55] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:31:56] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::f404:4f85:8ac5:2b16 : expressapisv2.net is 3.219.6.82, 3.211.
239.214, 3.220.254.12, 3.211.229.28, 3.209.45.230, 3.221.81.186
172.20.10.0/28 > 172.20.10.3  » [11:31:56] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::419:e243:455:c2ef : e86303.dscx.akamaiedge.net is 92.123.128.
189, 92.123.128.182, 92.123.128.181, 92.123.128.187, 92.123.128.185, 92.123.128.186, 92.123.128.180, 92.123.128.190, 92.123.128.191
172.20.10.0/28 > 172.20.10.3  » [11:32:00] [net.sniff.https] sni 172.20.10.2 > https://api.anthropic.com
[11:32:01] [net.sniff.https] sni 172.20.10.2 > https://www.expressapisv2.net
172.20.10.0/28 > 172.20.10.3  » [11:32:01] [net.sniff.https] sni 172.20.10.2 > https://api.anthropic.com
172.20.10.0/28 > 172.20.10.3  » [11:32:02] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::f404:4f85:8ac5:2b16 : expressapisv2.net is 3.219.6.82, 3.211.
239.214, 3.220.254.12, 3.211.229.28, 3.209.45.230, 3.221.81.186
172.20.10.0/28 > 172.20.10.3  » [11:32:03] [net.sniff.https] sni 172.20.10.2 > https://www.expressapisv2.net
172.20.10.0/28 > 172.20.10.3  » [11:32:03] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::f404:4f85:8ac5:2b16 : dns.msftncsi.com is 131.107.255.255
172.20.10.0/28 > 172.20.10.3  » [11:32:04] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::f404:4f85:8ac5:2b16 : a1961.g2.akamai.net is 23.200.147.185,
23.200.147.194
172.20.10.0/28 > 172.20.10.3  » [11:32:04] [net.sniff.dns] dns gateway > 172.20.10.2 : a1961.g2.akamai.net is 23.200.147.185, 23.200.147.194
172.20.10.0/28 > 172.20.10.3  » [11:32:04] [net.sniff.http.request] http 172.20.10.2 GET www.msftncsi.com/ncsi.txt
172.20.10.0/28 > 172.20.10.3  » [11:32:04] [net.sniff.http.response] http 23.200.147.185:80 200 OK -> 172.20.10.2 (14 B text/plain)
```

Figure 18. Sniff log showing Microsoft NCSI 200 OK response header, mail.google.com connections, Google Tag Manager, and Optimizely CDN traffic.

```
Microsoft NCSI

172.20.10.0/28 > 172.20.10.3  » [11:32:51] [net.sniff.https] sni 172.20.10.6 > https://a27160410180.cdn.optimizely.com
172.20.10.0/28 > 172.20.10.3  » [11:32:51] [net.sniff.https] sni 172.20.10.6 > https://a27160410180.cdn.optimizely.com
172.20.10.0/28 > 172.20.10.3  » [11:32:51] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:32:51] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:32:52] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:32:52] [net.sniff.https] sni 172.20.10.6 > https://www.bing.com
172.20.10.0/28 > 172.20.10.3  » [11:32:53] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e/pieceshash?c...
172.20.10.0/28 > 172.20.10.3  » [11:32:53] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e/pieceshash?c...
172.20.10.0/28 > 172.20.10.3  » [11:32:56] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::f404:4f85:8ac5:2b16 : onedscolprdwus11.westus.cloudapp.azure.
com is 20.189.173.12
172.20.10.0/28 > 172.20.10.3  » [11:32:56] [net.sniff.https] sni 172.20.10.2 > https://self.events.data.microsoft.com
172.20.10.0/28 > 172.20.10.3  » [11:32:58] [net.sniff.https] sni 172.20.10.2 > https://self.events.data.microsoft.com
172.20.10.0/28 > 172.20.10.3  » [11:33:00] [net.sniff.http.response] http 196.49.32.6:80 200 OK -> 172.20.10.6 (608 B application/octet-stream)
172.20.10.0/28 > 172.20.10.3  » [11:33:00] [net.sniff.http.response] http 196.49.32.6:80 200 OK -> 172.20.10.6 (608 B application/octet-stream)
172.20.10.0/28 > 172.20.10.3  » [11:33:00] [net.sniff.https] sni 172.20.10.2 > https://api.anthropic.com
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.dns] dns fe80::e49c:67ff:fe76:8d64 > fe80::419:e243:455:c2ef : a1856.dspw65.akamai.net is 23.36.15.248,
2.18.190.139
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.dns] dns gateway > 172.20.10.6 : a1856.dspw65.akamai.net is 23.36.15.248, 2.18.190.139
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.dns] dns gateway > 172.20.10.6 : a1856.dspw65.akamai.net is 23.36.15.248, 2.18.190.139
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.response] http 196.49.32.6:80 206 Partial Content -> 172.20.10.6 (1.8 kB application/octet-stream)
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e?P1=177505692...
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e?P1=177505692...
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.response] http 196.49.32.6:80 206 Partial Content -> 172.20.10.6 (2 B application/octet-stream)
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.response] http 196.49.32.6:80 206 Partial Content -> 172.20.10.6 (2 B application/octet-stream)
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e?P1=177505692...
172.20.10.0/28 > 172.20.10.3  » [11:33:02] [net.sniff.http.request] http 172.20.10.6 GET 196.49.32.6/filestreamingservice/files/ea3bc611-fa15-49e6-b10a-23b076
9c6a7e?P1=177505692...
```

Figure 19. Additional sniffed services showing Optimizely, Bing, Microsoft self.events.data services, and multiple HTTP 206 Partial Content responses.

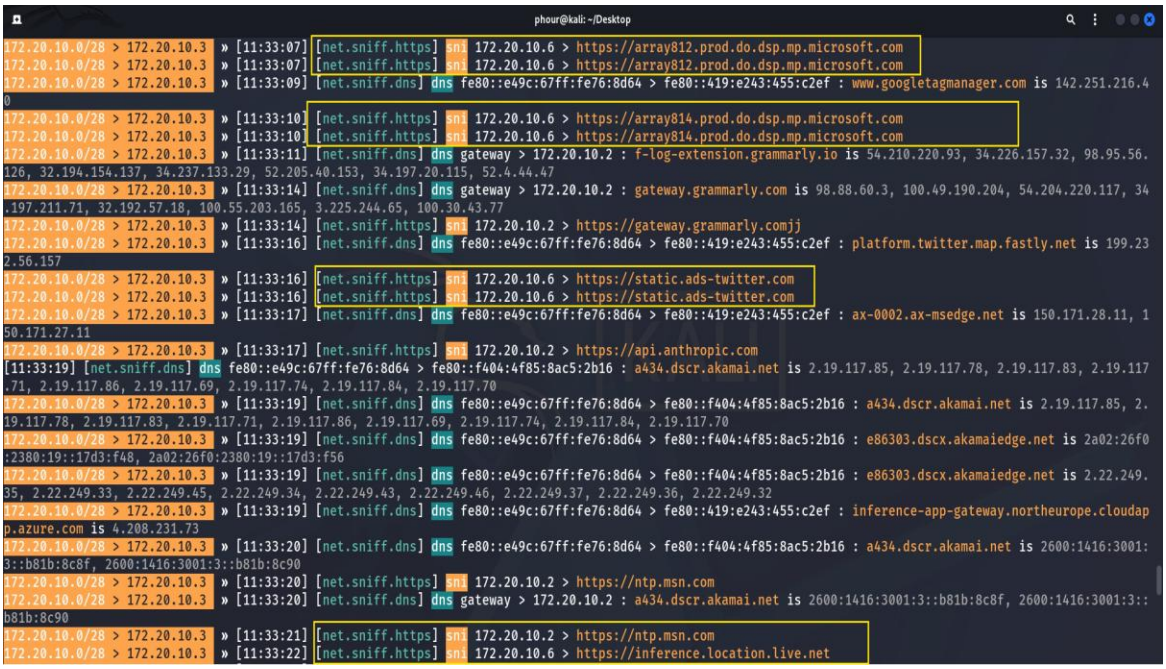

Figure 20. Further sniffed traffic including Microsoft arrayb12 update services, Grammarly extension connections, Twitter/X CDN, and MSN OTP services.

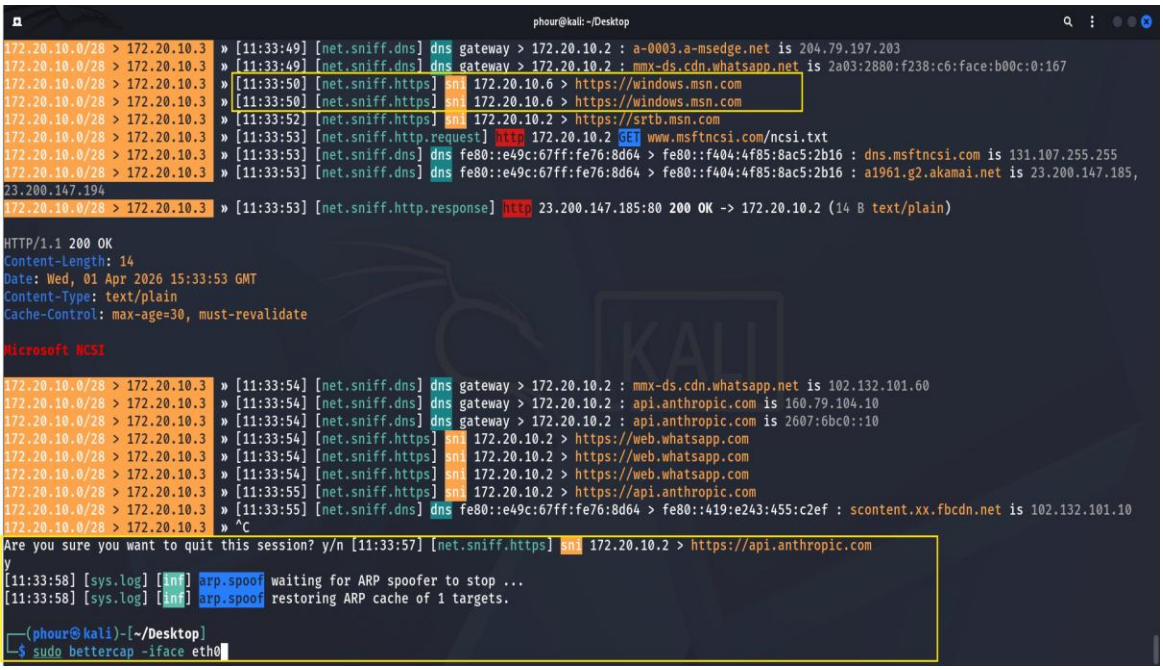

Figure 21. Final sniffing phase showing WhatsApp Web connections, session quit confirmation, and ARP cache restoration for target 172.20.10.6.

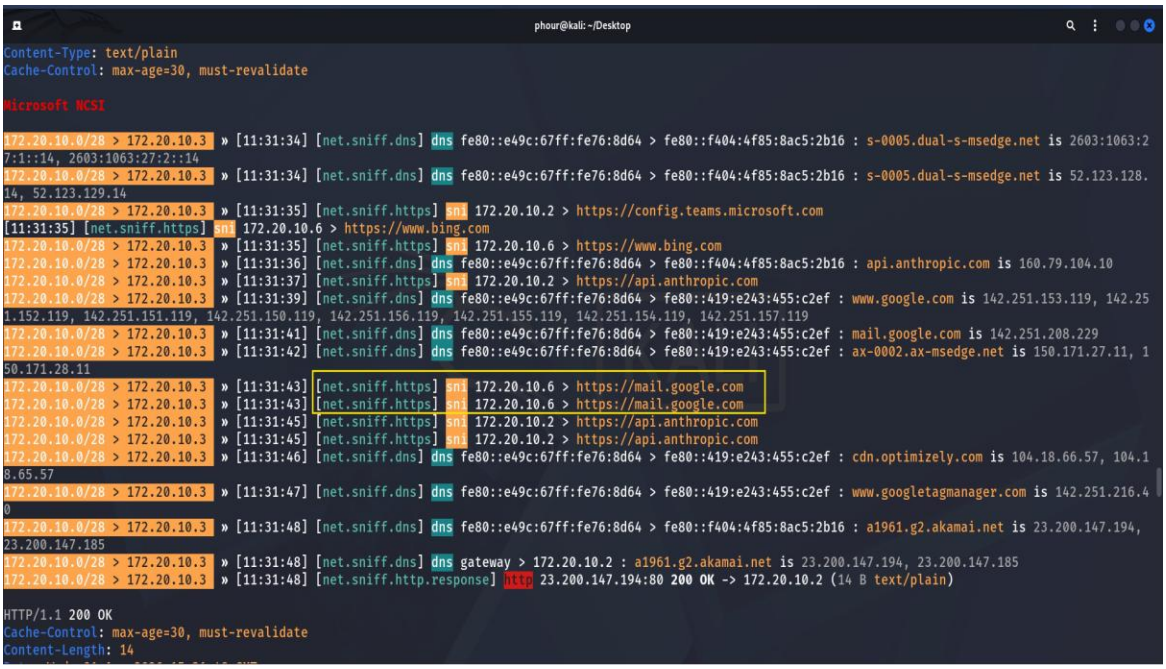

Figure 22. Post-session continuation showing mail.google.com connection attempts, Microsoft Teams config, and API service traffic captured.

Command Analysis

The following table summarizes every command used during the attack, what it does, why it was used, and what it produced:

Command	Function	Why Used	Output
sudo bettercap -iface eth0	Launches Bettercap with root privileges on the eth0 interface	eth0 is the wired Ethernet interface. Root is required for raw socket and ARP operations.	Bettercap starts, detects 172.20.10.0/28 subnet, gateway monitor starts.
help	Displays all available commands and lists all modules with their current status	Used to understand what is available and to confirm module status before starting.	Full module list shown, all "not running." Interactive command reference displayed.
help net.probe	Shows documentation and parameters for the net.probe module	Used before activation to understand all options and default values.	Module description and parameter list: mdns, nbns, throttle, upnp, wsd.

net.probe on	Activates the network probing module to discover live hosts	Active scanning is necessary to populate the ARP table with all reachable devices.	Three hosts discovered. net.recon auto-started. IPv6 addresses logged.
net.show	Prints a formatted ARP table of all discovered endpoints	Required to identify the target IP address and confirm the network layout.	Table with four hosts including gateway. Target 172.20.10.6 confirmed.
help arp.spoof	Shows documentation and parameters for the arp.spoof module	Used to review available parameters before configuring the attack.	Parameter list including fullduplex, internal, targets, and whitelist options.
set arp.spoof.fullduplex true	Enables bidirectional ARP poisoning of both target and gateway	Without full duplex, only one direction of traffic is intercepted. Full duplex gives complete MITM visibility.	Parameter saved. Bettercap warns about potential failure if router has ARP protection.
set arp.spoof.targets 172.20.10.6	Limits ARP poisoning to only the specified target IP	Restricts impact to one device, avoiding disruption of other hosts on the subnet.	Target set to 172.20.10.6. Only this device will be poisoned.
arp.spoof on	Starts sending forged ARP replies to target and gateway	This is the core MITM action. Places the attacker in the traffic path.	MITM position established. Log: "starting, probing 1 targets."
net.sniff on	Activates the packet capture and analysis module	With MITM position active, all traffic flows through the attacker. net.sniff reads and displays it.	Live capture begins. DNS, HTTPS SNI, HTTP responses all appear in the terminal.

Results and Interpretation

The attack was successful. The attacker achieved a full MITM position between the Windows 10 victim at 172.20.10.6 and the gateway at 172.20.10.3. All traffic flowing through the victim's machine during the session was intercepted and partially decoded.

DNS Interception

DNS queries are transmitted in plaintext over UDP port 53. Every domain the victim's browser and operating system resolved was captured in full, including the query, the server responding, and the IP addresses returned. A partial list of domains captured includes: gmail.com, mail.google.com, accounts.google.com, play.google.com, yahoo.com, tesla.com, facebook.com, bing.com, msn.com, edge.microsoft.com, web.whatsapp.com, api.anthropic.com, googletagmanager.com, grammarly.io, and twitter.com CDN addresses. This data alone builds a detailed profile of the victim's browsing habits and installed software.

HTTPS SNI Leakage

Despite HTTPS encrypting the content of all web sessions, the Server Name Indication (SNI) field in the TLS ClientHello message is transmitted unencrypted. The attacker captured HTTPS SNI entries confirming connections to accounts.google.com, mail.google.com, tesla.com, yahoo.com, login.live.com, web.whatsapp.com, and other encrypted services. The attacker cannot read the content of these sessions, but they can see exactly which sites the victim is visiting, when, and for how long.

HTTP Plaintext Capture

Several unencrypted HTTP exchanges were captured in full. The most consistent was the Microsoft Network Connectivity Status Indicator (NCSI) check, which Windows performs regularly by fetching msftncsi.com/ncsi.txt over plain HTTP. The full HTTP response was captured including headers: HTTP/1.1 200 OK, Content-Type: text/plain, Cache-Control: max-age=30 must-revalidate, and the Date header showing the exact time. Partial content (HTTP 206) responses from 196.49.32.6 were also captured, indicating file download activity.

Behavioral Impact on Victim

The attack had measurable effects on the victim's browsing experience. The Google account session was invalidated during the session, likely due to Google's detection of the changed network path. Facebook DNS resolution experienced repeated failures. The victim browsed Google, Yahoo, and Tesla, with Tesla and Yahoo loading successfully, while Facebook and some Google services

showed errors. These outcomes are consistent with how a real MITM attack affects victim connectivity: most traffic continues, but security-sensitive services detect anomalies.

Security Implications

This attack illustrates several significant security risks that apply to any device connected to an untrusted or shared network:

- ARP has no authentication. Any device can poison any ARP cache at any time. This is a protocol-level design limitation, not a software bug.
- DNS leaks browsing activity completely. Every domain a user visits is exposed to any attacker on the same network segment.
- HTTPS provides content confidentiality but not metadata privacy. The SNI field reveals destination domains to a MITM attacker even when content is encrypted.
- Background OS processes generate significant traffic. An attacker observing a victim's network traffic gains information about installed software, update behavior, and connected services without any user interaction.
- Browser extensions leak their own traffic. Grammarly, Google Cast, and similar extensions generated identifiable traffic that exposed software details about the victim's machine.

The applicable defenses are documented in the table below:

Defense	How It Mitigates the Attack
Dynamic ARP Inspection (DAI)	Implemented on managed switches. Validates ARP packets against a DHCP binding table. Forged ARP replies that do not match the table are dropped before reaching the target.
Static ARP Entries	For critical infrastructure devices, hardcoding the gateway MAC address into the ARP table prevents an attacker from overwriting it with a forged reply.
VPN	Routes all traffic through an encrypted tunnel to the VPN server before it hits the local gateway. A MITM attacker sees only encrypted VPN traffic with no readable metadata.

DNS over HTTPS (DoH) or DNS over TLS (DoT)	Encrypts DNS queries so domain lookups are not readable to a local attacker. The attacker would see encrypted UDP or TCP traffic instead of plaintext domain names.
Encrypted Client Hello (ECH)	Encrypts the SNI field in the TLS ClientHello, preventing the attacker from reading destination domains from HTTPS connections. Requires support from both browser and server.
Network Monitoring and ARP Watch Tools	Tools such as arpwatch, Zeek, or Snort can detect unusual ARP activity, such as frequent ARP replies not corresponding to requests, and alert administrators in real time.

Challenges and Resolution

Two notable issues were observed during the execution of this project:

Facebook DNS Failures

During the active sniffing phase, DNS queries for facebook.com and its subdomains repeatedly returned failed or incomplete results. On the victim machine, this manifested as a "can't reach this page" error. This was not caused by any intentional configuration in the attack setup. The most likely explanation is that ARP cache poisoning introduced latency or packet reordering in the DNS resolution path for Facebook's infrastructure, which uses complex DNS configurations with short TTLs and multiple servers. Facebook's infrastructure may also have network path validation that causes resolution failures when the packet route changes unexpectedly. This is a known side effect of ARP spoofing in some environments and does not indicate a failure of the tool or the technique.

Google Session Invalidation

The Google account on the victim machine showed a session expired message during the attack. Google's login infrastructure monitors session behavior including network path consistency. When the attacker's MITM position changed the routing of packets between the victim and Google's servers, Google's backend detected the anomaly and invalidated the active session as a security measure. This is expected behavior and serves as evidence that the attack was actively affecting the victim's experience. It also demonstrates that some services do have partial defenses against MITM positioning, though these defenses protect session state rather than traffic confidentiality.

Conclusion

This project successfully demonstrated a full ARP spoofing and packet sniffing attack using Bettercap in a controlled virtual lab environment. Starting from a Kali Linux terminal with no prior knowledge of the network, the attacker discovered all live hosts within 16 seconds using net.probe, identified the Windows 10 victim at 172.20.10.6, poisoned its ARP cache using full duplex arp.spoof, and captured all traffic from that device using net.sniff.

The results show that this attack requires no special hardware, no software installation on the victim machine, and no interaction from the victim. It takes four commands to execute after startup. The captured data included complete DNS query histories, HTTPS destination domains via SNI, full HTTP response headers, third-party tracking and analytics service traffic, browser extension connections, and OS background service activity.

The primary lesson is that network position equals intelligence. An attacker who can insert themselves between a device and its gateway can build a detailed behavioral profile of that device's user with minimal effort. Understanding this attack is the first step toward building networks that resist it. Defenders should deploy Dynamic ARP Inspection, encourage VPN use on shared networks, and promote adoption of DNS over HTTPS and Encrypted Client Hello as standard practices.

This project was executed solely for educational purposes in a private, isolated lab environment. No real user, network, or system was targeted.

Ibrahim Ayomide Fayomi | fayomiibrahim44@gmail.com | linkedin.com/in/fayomi31/